

# Socle commun technique

*Règles applicables à TOUS les cahiers des charges (M1)*

DOCUMENT

**Socle technique**

PROMOTION

**Master 1**

## Sommaire

---

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>Règles applicables à TOUS les cahiers des charges (M1)</b> | <b>3</b> |
| 1. L'environnement fourni                                     | 3        |
| 2. Traduction des « zones » du CDC en réseaux Docker          | 3        |
| Règles de mise en œuvre                                       | 4        |
| 3. Publication et filtrage sur l'OPNsense                     | 4        |
| 4. Détection : Suricata et EveBox                             | 4        |
| 5. Nommage                                                    | 5        |
| 6. Services vulnérables                                       | 5        |
| Niveaux                                                       | 5        |
| 7. Flags                                                      | 6        |
| 8. Réalisme (critère noté)                                    | 6        |
| 9. Documentation                                              | 7        |
| Document Pédagogique (PDF, complet)                           | 7        |
| Document Attaquant (PDF, minimal)                             | 7        |
| 10. Contraintes d'exploitation                                | 7        |

## Règles applicables à TOUS les cahiers des charges (M1)

Ce document complète chaque CDC. En cas de contradiction, **le CDC prime** sur le socle pour le *besoin métier*, et **le socle prime** sur le CDC pour la *mise en œuvre technique*.

### 1. L'environnement fourni

Chaque groupe reçoit une configuration **WireGuard (LabAuto)** donnant accès à un environnement déjà monté. **Vous ne l'installez pas : vous le configurez et vous déployez dedans.**

| Machine                 | Adresse       | Rôle                                                            | Fournie |
|-------------------------|---------------|-----------------------------------------------------------------|---------|
| Jumpbox                 | 10.85.4.20    | point d'entrée SSH, rebond vers l'environnement                 | Oui     |
| Routeur <b>OPNsense</b> | 192.168.10.1  | routage, filtrage, NAT, publication des services (web sur 443 ) | Oui     |
| Sonde <b>Suricata</b>   | 192.168.10.30 | capture réseau, interface <b>EveBox</b> sur 5636                | Oui     |
| Hôte <b>Docker</b>      | 192.168.10.50 | hôte de tous vos services, interface sur 5001                   | Oui     |

Accès depuis votre poste (voir aussi §7.5 de l'énoncé) :

```
ssh -N -L 8443:192.168.10.1:443 -L 5636:192.168.10.30:5636 -L 5001:192.168.10.50:5001
etudiant@10.85.4.20
ssh -J etudiant@10.85.4.20 etudiant@192.168.10.50 # hôte Docker
ssh -J etudiant@10.85.4.20 etudiant@192.168.10.30 # sonde Suricata
```

Une **notice** fournie par l'équipe pédagogique détaille le déploiement des services, le routage et le filtrage, la détection Suricata et la publication des services vulnérables. Ce socle décrit **ce qui est attendu**, la notice décrit **comment le faire**.

### 2. Traduction des « zones » du CDC en réseaux Docker

Chaque CDC décrit six **zones logiques**. Sur l'hôte Docker fourni, ces zones sont des **réseaux Docker distincts**, non routés entre eux par défaut.

| Zone du CDC | Réseau Docker | Sous-réseau conseillé | Rôle                                               |
|-------------|---------------|-----------------------|----------------------------------------------------|
| Publication | net-dmz       | 172.30.10.0/24        | services publiés vers l'extérieur                  |
| Serveurs    | net-srv       | 172.30.20.0/24        | services applicatifs internes                      |
| Données     | net-data      | 172.30.30.0/24        | bases de données, stockage, sauvegarde             |
| Postes      | net-users     | 172.30.40.0/24        | postes ou conteneurs représentant les utilisateurs |

|                 |                        |                             |                                                                                 |
|-----------------|------------------------|-----------------------------|---------------------------------------------------------------------------------|
| Administration  | <code>net-admin</code> | <code>172.30.50.0/24</code> | outils d'exploitation et d'administration                                       |
| Zone spécifique | <code>net-spec</code>  | <code>172.30.60.0/24</code> | zone métier du CDC (OT, biomédical, CI/CD, magasins, vidéo, recherche, clients) |

Un CDC peut exiger une séparation supplémentaire à l'intérieur d'une zone (réseau Wi-Fi public d'une collectivité, réseau étudiant d'une université) : créez alors un réseau Docker dédié ( `net-users-pub` , `net-users-etu` ) et documentez-le.

### Règles de mise en œuvre

- **Un service = un conteneur**, attaché à **un seul** réseau, sauf s'il joue explicitement un rôle de passerelle prévu par le CDC (proxy, passerelle de maintenance, point de rebond). Un conteneur attaché à deux réseaux est un **chemin d'attaque** : c'est un choix, pas un accident.
- **Aucun `network_mode: host`**, **aucun `--privileged`**, **jamais** de montage de `/var/run/docker.sock` — sauf si ce montage est justement la vulnérabilité de niveau N3 choisie, et il est alors documenté.
- Les ports ne sont publiés sur l'hôte ( `ports:` ) **que** pour les services que le CDC marque « publié ». Tous les autres n'exposent leurs ports que sur leur réseau Docker ( `expose:` ).
- `restart: unless-stopped` sur tous les services : le SI doit se relever seul.
- Un fichier `docker-compose.yml` par zone (ou un fichier global avec des profils), versionné et commenté.

## 3. Publication et filtrage sur l'OPNsense

- L'OPNsense est la **seule** porte d'entrée vers votre SI.
- Seuls les services marqués « publié » dans le CDC font l'objet d'une redirection (NAT port-forward) vers l'hôte Docker.
- Les autres ports de l'hôte Docker ne sont pas atteignables depuis l'extérieur.
- Règles minimales attendues :
  - WAN → hôte Docker : uniquement les ports des services publiés ;
  - toute autre entrée : refusée ;
  - règle terminale : **refus journalisé**.
- Chaque règle est justifiée dans la documentation : source, destination, port, motif métier.
- Les journaux du pare-feu sont conservés et exploitables pendant la phase 2.

Un SI où tous les conteneurs publient leurs ports sur l'hôte est **éliminatoire** sur le critère de segmentation : l'attaquant obtient tout le SI d'un seul `nmmap`.

## 4. Détection : Suricata et EveBox

La sonde est fournie et capture le trafic. **Ce qui est attendu de vous, c'est de l'exploiter.**

- Les règles existantes se trouvent dans `/var/lib/suricata/rules/suricata.rules` (jeu ET/Open télé-chargé au build).
- **Vos règles vont dans `/etc/suricata/rules/tp-local.rules`**, avec des `sid` à partir de **9000001**.
- L'interface **EveBox** ( `https://127.0.0.1:5636` via le tunnel) sert à consulter les alertes et les événements.

- La cellule défense peut — et devrait — préparer ses règles **pendant la phase de build** : vous connaissez vos chemins d'attaque, donc vous connaissez les signaux.
- Les journaux applicatifs des conteneurs doivent rester consultables (`docker compose logs`, ou montage des journaux dans un volume) : ils constituent la seconde source de preuve du rapport de détection.

Familles de détection à couvrir (détail dans le *Guide builder*, §4) : reconnaissance, exploitation initiale, mouvement latéral, franchissement de la zone spécifique.

## 5. Nommage

Format : `<code-service>-<zone><nn>` — exemple : `web-dmz01`, `gmao-srv02`, `db-data01`.

- Le **domaine fictif** de l'entreprise est imposé par votre cahier des charges (un domaine interne en `.lan` et un domaine public).
- Les bannières, pieds de page, certificats auto-signés, comptes et adresses de messagerie utilisent ce domaine.
- Un conteneur nommé `vulhub-web-1` ou une page « It works! » = malus réalisme.

## 6. Services vulnérables

- Source unique : dépôt **Vulhub** (<https://github.com/vulhub/vulhub>).
- Le CDC impose la **fonction métier**, la **zone** et le **niveau d'exposition** de chaque service. **Le choix des services vulnérables, des vulnérabilités et de leur profondeur appartient au groupe** : c'est votre valeur ajoutée d'intégrateur, et c'est ce que l'attaquant devra découvrir. Piochez dans l'*annexe 1 — Catalogue Vulhub*.
- Quotas applicables à **tous** les CDC : **12 services vulnérables** (2 × 6 membres), dont **3 points d'entrée N1** ; les **9 autres se répartissent librement entre N2 et N3**, et doivent former **au moins 2 chaînes complètes N1 → N2 → N3**. Chaque CDC est dimensionné pour cela : 16 ou 17 services en tranche ferme, dont l'OPNsense fourni, soit 15 ou 16 services déployés — 12 vulnérables et 3 ou 4 sains. Méthode détaillée : *Guide builder*.
- **Toute modification est autorisée et encouragée** (thème, données, comptes, contenu, reverse proxy devant le service, port non standard, bannière) tant que la vulnérabilité reste exploitable de bout en bout.
- **Interdit** : patcher la vulnérabilité, filtrer l'URL vulnérable, mettre un WAF bloquant, exiger une authentification non fournie pour un service de niveau 1.
- Chaque service vulnérable doit être **testé et validé** par le groupe (preuve dans le document pédagogique : commande + sortie).

### Niveaux

| Niveau | Définition                                                                                                      | Accessible depuis                                                    |
|--------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| N1     | Point d'entrée. Exploitable directement depuis l'extérieur avec les seuls accès initiaux du Document Attaquant. | WAN, via l'OPNsense                                                  |
| N2     | Exploitable seulement depuis un conteneur compromis en N1 (rebond réseau, credentials trouvés en N1).           | <code>net-dmz</code> / <code>net-srv</code>                          |
| N3     | Exploitable seulement après un N2. Cœur du SI (données, zone spécifique, administration).                       | <code>net-srv</code> / <code>net-data</code> / <code>net-spec</code> |

Les trois N1 sont les points d'entrée du SI, et les seuls. C'est la règle structurante :

- exactement 3 **services vulnérables** sont atteignables et exploitables avec les seuls accès initiaux du Document Attaquant ;
- **aucune autre vulnérabilité** ne doit être exploitable directement depuis l'extérieur — si un quatrième service se révèle attaquant sans pivot, il est requalifié N1 et vous êtes hors quota ;
- les 9 **vulnérabilités restantes** se répartissent comme vous le souhaitez entre N2 et N3, du moment qu'elles forment **au moins 2 chaînes complètes** N1 → N2 → N3.

Un service annoncé N1 qui n'est pas réellement atteignable avec les accès initiaux est requalifié en N2/N3 (et la documentation corrigée).

## 7. Flags

- Les flags proviennent **exclusivement** du CSV fourni par l'équipe pédagogique.
- **1 flag par service vulnérable minimum** (soit 12), plus 1 flag par pivot notable (accès base de données, obtention d'un compte privilégié, franchissement de zone).
- Placement selon le type d'exploitation :

| Type d'exploitation     | Emplacement du flag                                         |
|-------------------------|-------------------------------------------------------------|
| RCE / reverse shell     | /root/flag.txt ou /flag.txt                                 |
| LFI / path traversal    | /var/www/html/flag.txt ou /etc/flag                         |
| SQLi                    | table <code>flags</code> , colonne <code>flag_string</code> |
| FTP / SMB / NFS         | /srv/ftp/flag.txt ou racine du partage                      |
| Appli web sans accès FS | profil utilisateur, document métier, ticket, note           |
| Windows                 | C:\flag.txt ou %SYSTEMROOT%\Temp\flag.txt                   |

- Un flag doit être **atteignable uniquement par l'exploitation prévue** : pas de flag dans une page publique, pas de flag dans un répertoire listable, pas de flag dans une image publique ni dans un dépôt Git accessible.
- Attention aux flags perdus au redémarrage : un flag écrit dans un conteneur sans volume disparaît au `docker compose down`. Placez-les via le `Dockerfile`, un script d'init ou un volume.
- La correspondance flag ↔ service ↔ chemin d'exploitation est consignée dans l'*annexe 3 — Matrice des flags* et dans le document pédagogique — **jamais** ailleurs. Le modèle est pré-rempli avec un exemple générique : 3 flags N1 et 9 répartis entre N2 et N3, à remplacer par vos propres services et vos propres valeurs.

## 8. Réalisme (critère noté)

Chaque SI doit « raconter » son entreprise. Minimum attendu :

- **Données métier** : les volumes exigés par le §6 de votre CDC (patients, commandes, ordres de fabrication, dossiers citoyens, tickets...). Données **factives** — aucune donnée personnelle réelle.
- **Comptes** : l'annexe du CDC liste les comptes attendus. Mots de passe de qualité variable, avec au moins un compte faible **justifié** (compte de test, prestataire, ancien stagiaire).
- **Traces d'usage** : historique de commandes, messages internes, documents partagés, tickets ouverts, notes de procédure. C'est là que se cachent les credentials de pivot.
- **Bannières et versions** cohérentes avec l'âge supposé du service.

---

## 9. Documentation

---

### Document Pédagogique (PDF, complet)

1. Présentation de l'entreprise fictive et rappel du CDC choisi.
2. Schéma réseau complet : conteneurs, réseaux Docker, OPNsense, sonde, flux.
3. Tableau d'inventaire : nom du conteneur, réseau, image, version, ports, exposition.
4. Pour **chaque service vulnérable** : image Vulhub utilisée, CVE, niveau, modifications apportées, vecteur d'attaque, exploit validé (commande + preuve), emplacement du flag, preuve attendue de l'attaquant.
5. Chaînes de dépendance (schéma).
6. Comptes et accès complets.
7. Configuration OPNsense (règles de publication et de filtrage) et règles Suricata écrites.
8. **Tableau de répartition des tâches** nominatif.

### Document Attaquant (PDF, minimal)

1. Contexte de la mission (présentation de l'entreprise fictive, périmètre autorisé).
2. Schéma réseau **partiel** : uniquement le point d'entrée et les services publiés.
3. Accès initiaux : URL et ports publiés, et le cas échéant un compte utilisateur de bas privilège.
4. Règles d'engagement et contacts.

**Ne doit PAS y figurer** : liste des CVE, plan des réseaux Docker internes, credentials internes, emplacement des flags, schéma des zones internes.

---

## 10. Contraintes d'exploitation

---

- Tous les services de la tranche ferme tournent **en même temps** sur l'hôte Docker : surveillez la RAM. Préférez les images légères, limitez les services Java lourds (WebLogic, Confluence, Jira) à deux ou trois maximum, et déclarez des limites ( `mem_limit` ) sur les gourmands.
- Conflits de ports : plusieurs environnements Vulhub utilisent `8080` . Publiez chacun sur un port distinct et faites la traduction dans le reverse proxy.
- Sauvegardez votre configuration : `docker-compose.yml` , `Dockerfile` , scripts d'init et jeux de données doivent être **versionnés** et rejouables. Un SI qu'on ne sait pas redéployer est un SI perdu.
- Avant le début de la phase 2 : `docker compose down && docker compose up -d` complet, puis re-vérification des chaînes. Ce qui ne survit pas au redémarrage n'existe pas.